

Data & Transaction Management in MSA Architecture

Vraj Suratwala

MSA

Department of ICT, VNSGU

2.1 Database and MSA

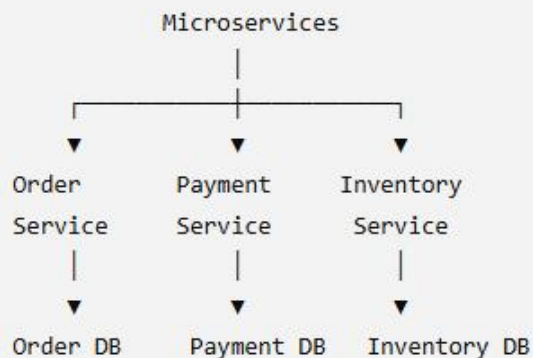
Connection of Data with Micro Services

Introduction

- In Microservices Architecture (MSA), database management is different from a traditional monolithic application because each microservice is generally responsible for its own persistent data.
- The reference book describes this as the Database per Service pattern, where a service's database is private and accessible only through the service's API.

1. Database per Service

Each microservice owns and manages its own data.



The database is considered part of the **implementation of the service**. Other services should not directly access it; they should communicate through the service's API. [msabook](#)

Ways to Implement Database per Service

For relational databases, the reference book identifies three approaches: [msabook](#)

1. **Private Tables per Service** – Each service owns a specific set of tables.
2. **Schema per Service** – Each service has its own private database schema.
3. **Database Server per Service** – Each service has its own database server.

Private tables and **schema per service** generally have lower overhead, while high-throughput services may require their own database server. [msabook](#)

2. Different Databases for Different Services

- One advantage of MSA is that different services can select the database technology best suited to their requirements.
- For example:
 - Order Service → Relational Database
 - Product Service → MongoDB
 - Social Graph → Neo4j
 - Text Search Service → Elasticsearch

3. Shared Database

- Sometimes multiple services may use a shared database, particularly when:
- A transaction spans multiple services.
- Operations require data from multiple services.
- Database replication is required.
- Service A —┐
- |
- Service B —┴—→ Shared Database
- |
- Service C —┐
- However, shared databases increase coupling because services may become dependent on the same database structure.

4. Main Database Problem in MSA

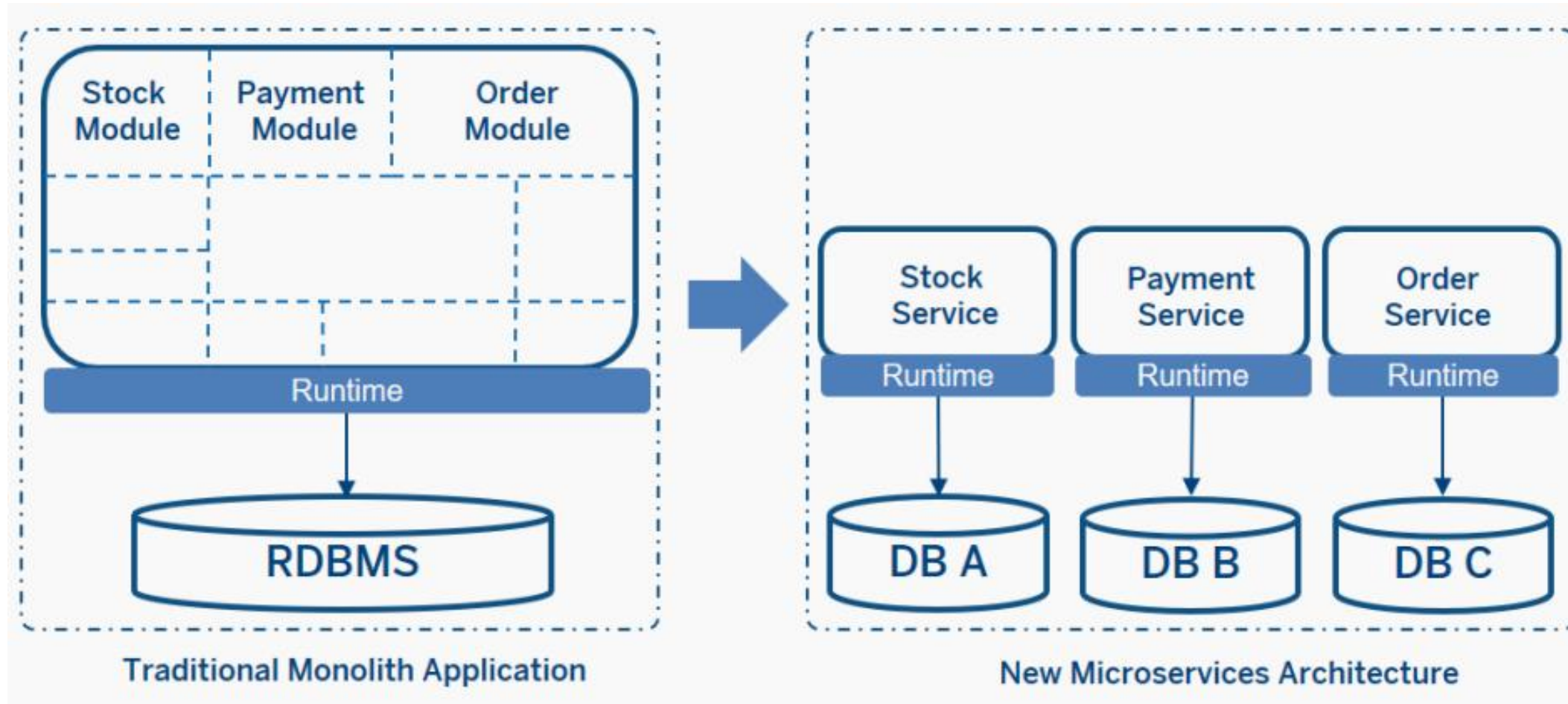
- In a monolithic application, a shared relational database can provide ACID transactions across business operations.
- In MSA, each service may have its own database, so there is:
- No central database and no single unit of work across all services.
- A business operation may therefore involve several local transactions across different services

5. Solutions for Database Challenges

<u>Problem</u>	<u>Possible Solution</u>
Transaction across services	Saga Pattern
Query across services	API Composition
Query across services	CQRS
Data synchronization	Events / Eventual Consistency

- PROS :
 - Loose Coupling: Changes to one service's database do not directly affect other services.
 - Independent Technology: Each service can choose a suitable database.
 - Independent Scaling: Database resources can be scaled according to individual service requirements.
 - Better Encapsulation: Other services cannot directly access internal data.
-
- CONS :
 - Transactions across multiple services become difficult.
 - Queries involving multiple databases are challenging.
 - Managing multiple SQL and NoSQL databases increases operational complexity.

Diagram



2.2 ACID vs BASE Transactions

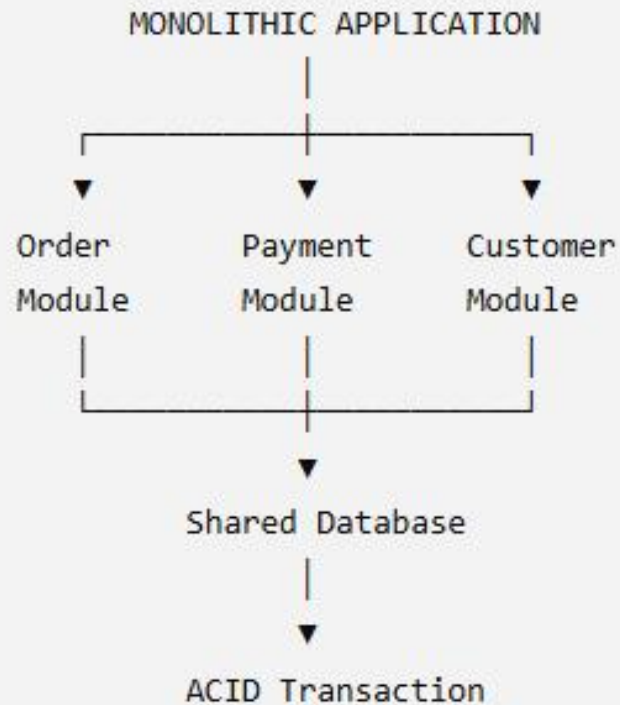
Important Concept

ACID Properties

- A – Atomicity
 - A transaction is treated as a single unit.
 - Either all operations succeed or all operations fail.
 - There should be no partial transaction state
 - Example:
 - If money is transferred from Account A to Account B, both debit and credit must succeed. If one fails, the transaction is rolled back.
- C – Consistency
 - The database must remain in a valid and consistent state before and after the transaction.
 - All defined database rules and constraints must be maintained.
- I – Isolation
 - Multiple transactions executing at the same time should not improperly interfere with each other.
 - One transaction's intermediate state should not incorrectly affect another transaction.
- D – Durability
 - Once a transaction is successfully committed, its changes are permanently stored even if a system failure occurs.
 - Your PPT summarizes ACID as: Atomicity, Consistency, Isolation, and Durability

ACID in Monolithic Applications

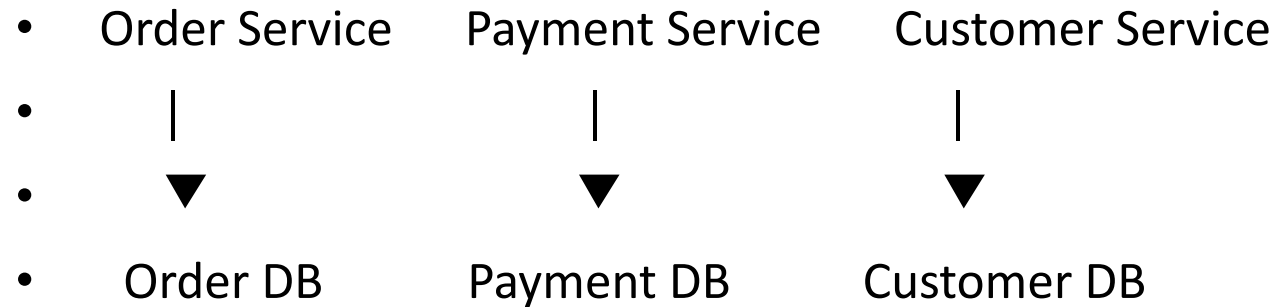
In a monolithic application, services/modules commonly use a **shared relational database**.



A single database can manage the complete transaction using ACID properties.

Problem with ACID in MSA

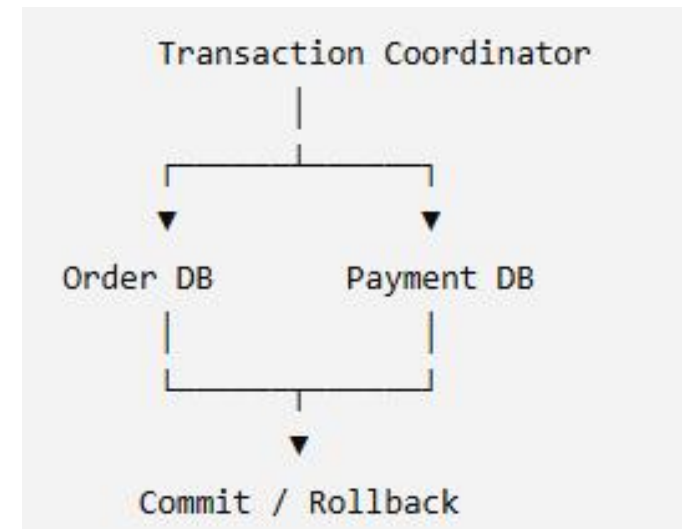
- In Microservices Architecture, each service generally has its own data store.



- Therefore:
- There is no central database.
- There is no single unit of work across all databases.
- One business operation may involve multiple local transactions.
- The PPT explicitly states that a single transaction unit of work cannot be used across multiple databases in a microservices architecture.

Distributed Transactions

- One possible solution is a distributed transaction, where a transaction involves two or more resources such as databases or message queues.
- A transaction coordinator manages the transaction across these resources.
- Two-Phase Commit (2PC)
- 2PC is a blocking protocol that ensures participating transactions either all succeed or all fail together.
- However, the PPT states that distributed transactions are complex, can cause high latency and low throughput, may experience deadlocks, and the transaction coordinator can become a single point of failure. It also notes that 2PC is not suitable for microservice architecture



BASE Transactions

- BASE is an alternative model used with distributed systems and eventual consistency.
- BASE stands for:
 - B – Basically Available
 - S – Soft State
 - E – Eventual Consistency

- B – Basically Available
- The system prioritizes availability of data. Data may be replicated across nodes so that the system remains available even when some nodes have problems.
- S – Soft State
- The state of the system can change over time, even without a new request, because updates may be propagated asynchronously.
- Consistency is not necessarily maintained immediately.
- E – Eventual Consistency
- Data may temporarily be inconsistent across services, but the system is designed so that the data becomes consistent eventually.
- eventual consistency allows short-term inconsistency while distributed data is being synchronized.

ACID vs BASE

Feature	ACID	BASE
Full Form	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft State, Eventual Consistency
Main Goal	Strong consistency	High availability and eventual consistency
Consistency	Immediate/strong	Eventual
Availability	May be reduced during transaction coordination	Prioritized
Transaction	Atomic transaction	Often uses local/asynchronous transactions
Typical Use	Conventional relational systems	Distributed/microservices systems
Failure Handling	Rollback	Often uses asynchronous recovery/compensation
MSA Suitability	Difficult across independent databases	More suitable for distributed MSA scenarios

2.3 Event Sourcing

pattern used in Microservices Architecture

Introduction

- Event Sourcing is a pattern used in Microservices Architecture where the state of an application is stored as a sequence of events rather than storing only the latest state of the data.
- Instead of simply storing:
- Order Status = Confirmed
- the system stores the events that produced that state:
- Order Created
- ↓
- Payment Completed
- ↓
- Order Confirmed
- The current state can then be reconstructed by replaying these events.

1. Basic Concept

In a traditional database approach, the current state is stored directly.

Traditional Approach



Order

- |— Order ID: 101
- |— Customer: Vraj
- |— Status: Confirmed

With **Event Sourcing**, the system stores the sequence of changes/events:

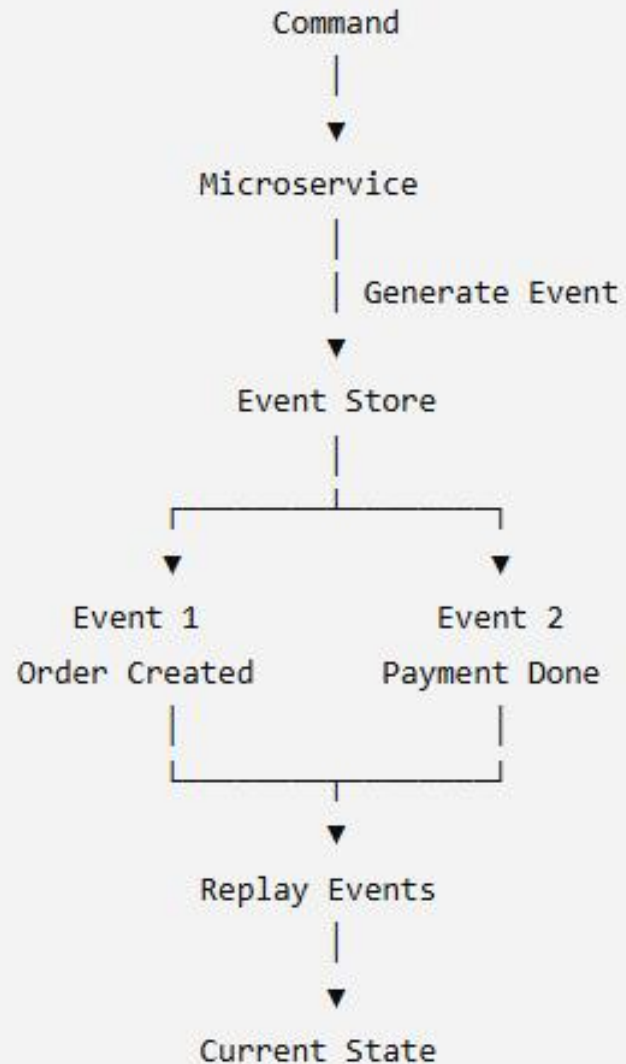
Event Store



- Event 1 → Order Created
- Event 2 → Payment Completed
- Event 3 → Order Confirmed

The current state is derived from these events.

2. Event Sourcing Flow



3. Event Store

The Event Store is the storage location containing the sequence of events.

Events are generally append-only records.

For example:

Event Store

1. OrderCreated
2. ItemAdded
3. PaymentCompleted
4. OrderConfirmed

Rather than overwriting previous information, new events are added to the event history.

- **4. Reconstructing Current State**

- The current state can be obtained by replaying the stored events in their correct order

- Example

- Initial State

- ↓

- OrderCreated

- ↓

- ItemAdded

- ↓

- PaymentCompleted

- ↓

- OrderConfirmed

- ↓

- Current Order State

- Therefore:

- Current State = Result of Applying All Relevant Events

- This is particularly useful when the history of changes is important.

5. Event Sourcing in MSA

Event Sourcing can be useful in MSA because services maintain their own data and can publish events when their data changes.



The reference material identifies **Event Sourcing** as one way of reliably updating data and publishing events when maintaining data consistency across services. [msabook](#)

6. Event Sourcing and Eventual Consistency

- Event Sourcing is closely related to event-driven communication and can support eventual consistency.
- For example:
 - Order Service
 - |
 - | Order Created Event
 - ▼
 - Customer Service
 - |
 - | Update
 - ▼
 - Customer Data
- The services may not update at exactly the same time. Instead, the event is propagated and consumed, allowing the system to become consistent over time.

Advantages

- 1. Complete History
 - All important changes are preserved as events.
- 2. Auditability
 - The system can determine what happened and when.
- 3. State Reconstruction
 - The current state can be reconstructed by replaying events.
- 4. Event-Based Integration
 - Events can be consumed by other services to update their own data.
- 5. Supports Event-Driven Architecture
 - Events can be used as a communication mechanism between services.

Challenges

- 1. Increased Complexity
- 2. Eventual Consistency
- 3. Query Difficulty
- 4. Event Versioning
- 5. Storage Growth

2.4 Eventual Consistency

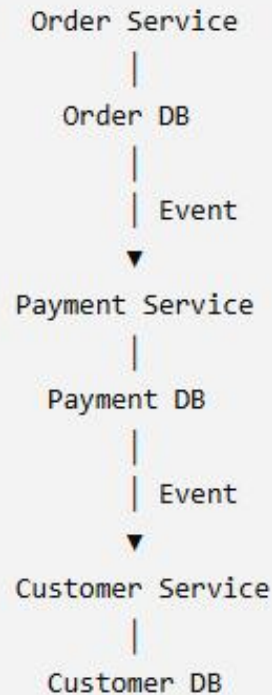
consistency model used in distributed systems

Introduction

- Eventual Consistency is a consistency model used in distributed systems where all copies of data may not become consistent immediately, but are expected to become consistent eventually after updates are propagated.
- In Microservices Architecture, different services often maintain separate databases. Therefore, when one service updates its data, other services may receive the update asynchronously and temporarily contain an older version.

2. Why Eventual Consistency is Used in MSA?

In MSA, services are independently deployed and commonly have their **own databases**.



A single ACID transaction cannot normally span these independent databases. Therefore, MSA often uses **local transactions + asynchronous communication**.

Your PPT explains that eventual consistency allows a short period of inconsistency while distributed data is being synchronized. [msa-presentation](#)

Working of Eventual Consistency

Consider an order:

Order Service

|
| Update Order



Order DB

|
| Publish Event



Payment Service

|
| Process Event



Payment DB

The update does not necessarily reach every service at the same instant.

Example

Initially:

Order Status = Pending

Order Service changes it to:

Order Status = Confirmed

For a short period:

Order DB → Confirmed

Payment DB → Pending

Customer DB → Pending

After events are processed:

Order DB → Confirmed

Payment DB → Confirmed

Customer DB → Confirmed

Thus, the system eventually reaches a consistent state.

Comparision

4. Eventual Consistency vs Strong Consistency

Strong Consistency

Data is immediately consistent

Updates are visible immediately

Strong transaction guarantees

More coordination may be required

Common in traditional transactional systems

Eventual Consistency

Data may temporarily be inconsistent

Updates propagate over time

Often uses asynchronous processing

Less coordination between services

Common in distributed/Microservices systems

Pros & Cons

- Pros
 - 1. High Availability
 - 2. Better Scalability
 - 3. Loose Coupling
 - 4. Suitable for Distributed Systems
- Cons
 - Temporary inconsistency
 - Asynchronous failures:
 - More complex application logic
 - Debugging difficulty:

Relationship with SAGA

- Eventual consistency is closely related to the Saga Design Pattern.
- A distributed business transaction is divided into multiple local transactions. Each local transaction updates its own database and triggers the next step.
- Local Transaction A
 - ↓
 - Event
 - ↓
- Local Transaction B
 - ↓
 - Event
 - ↓
- Local Transaction C
 - ↓
- Eventually Consistent State

2.5 Saga Design Pattern

use to manage business transactions that span multiple microservices.

Introduction

- The Saga Design Pattern is used to manage business transactions that span multiple microservices. Instead of using one large distributed transaction, a Saga divides the transaction into a sequence of local transactions, where each service updates its own database. If a step fails, previously completed steps are undone using compensating transactions.
- Your MSA material describes Saga as a pattern that works with eventual consistency, where a business transaction is divided into local transactions coordinated through events or messages

Need of SAGA

- In a monolithic application, multiple operations can often be handled within one database transaction.

- Monolithic Application



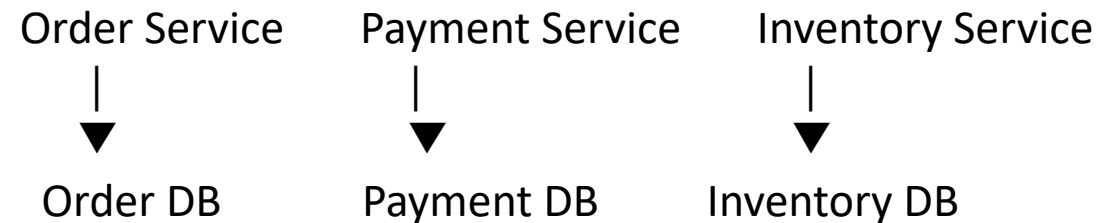
- Shared Database



- Single ACID

- Transaction

In MSA, services generally have separate databases:



A single ACID transaction cannot normally cover all these independent databases.

Therefore, Saga is used to manage the distributed business transaction.

Basic Idea of Saga

- A Saga consists of a sequence of local transactions.

- Local Transaction 1

- ↓

- Event

- ↓

- Local Transaction 2

- ↓

- Event

- ↓

- Local Transaction 3

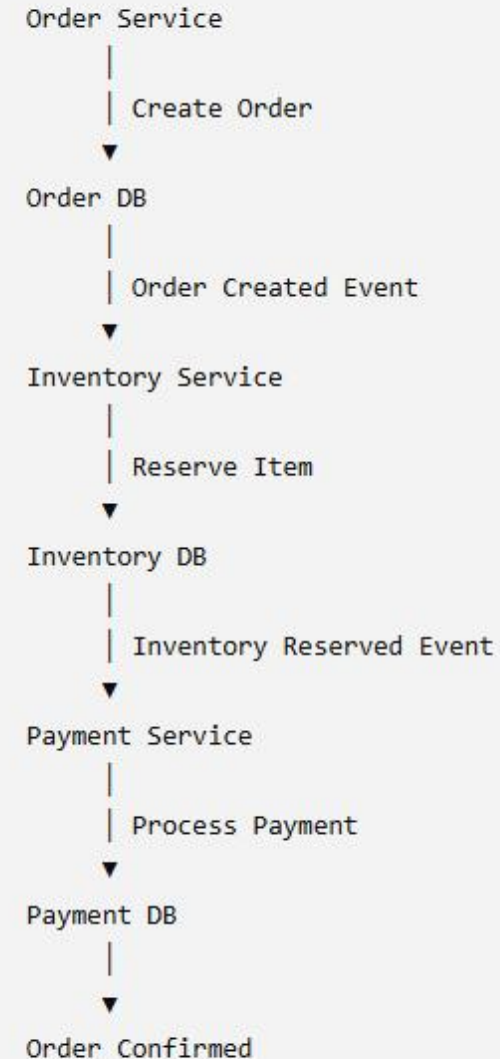
Each local transaction:

- Updates its own database.
- Commits independently.
- Triggers the next step.

If all steps succeed, the Saga completes successfully.

- Example – Online Order
- Consider placing an online order.
- The transaction may contain:
 - 1. Create Order
 - 2. Reserve Inventory
 - 3. Process Payment
 - 4. Confirm Order

Successful Saga



Each service performs its own **local transaction**.

What Happens When a Step Fails?

- Suppose payment fails after the order has already been created and inventory has been reserved.
- Create Order ✓
- ↓
- Reserve Inventory ✓
- ↓
- Process Payment ✗
-
- The system must compensate for the successful operations:
- Payment Failed
- ↓
- Release Inventory
- ↓
- Cancel Order
- These reverse actions are called Compensating Transactions.

Compensating Transaction

- A compensating transaction is an operation that semantically reverses the effect of a previously completed local transaction.

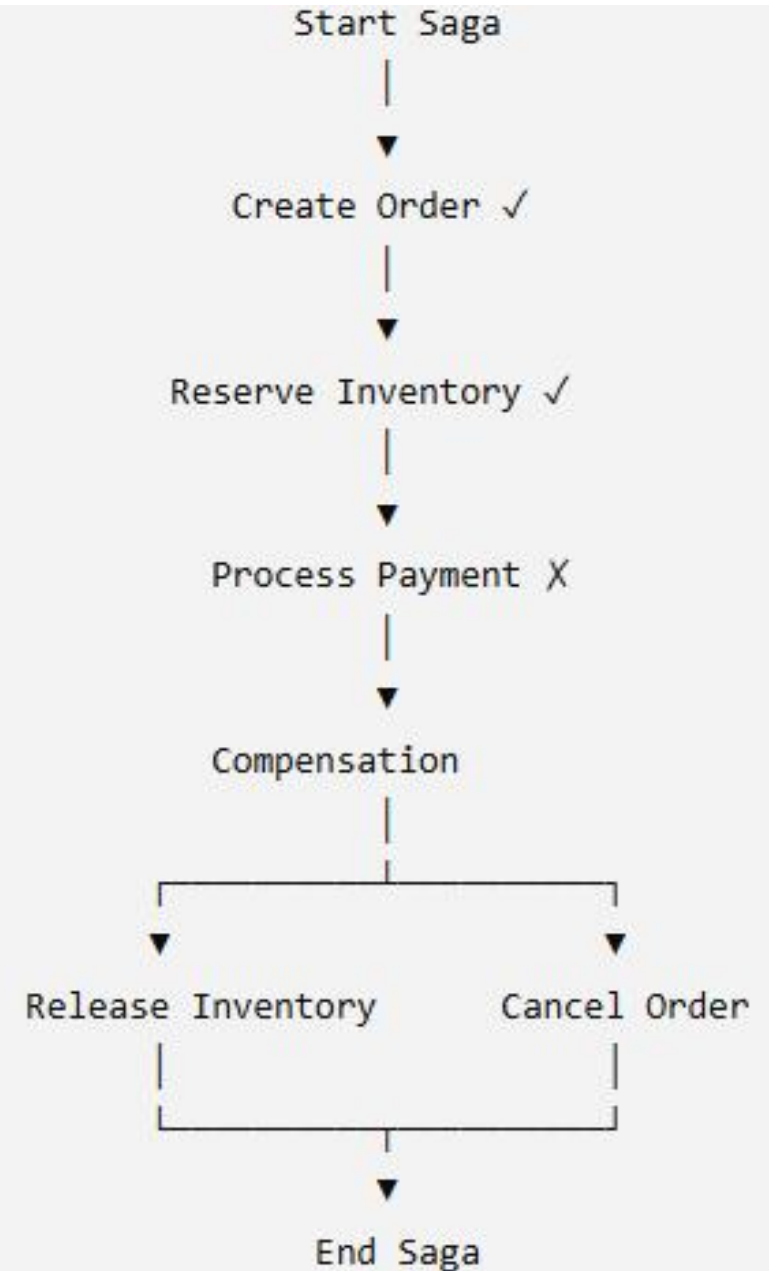
Original Transaction	Compensation
Create Order	Cancel Order
Reserve Inventory	Release Inventory
Charge Payment	Refund Payment

Important

A compensation is **not necessarily a database rollback**.

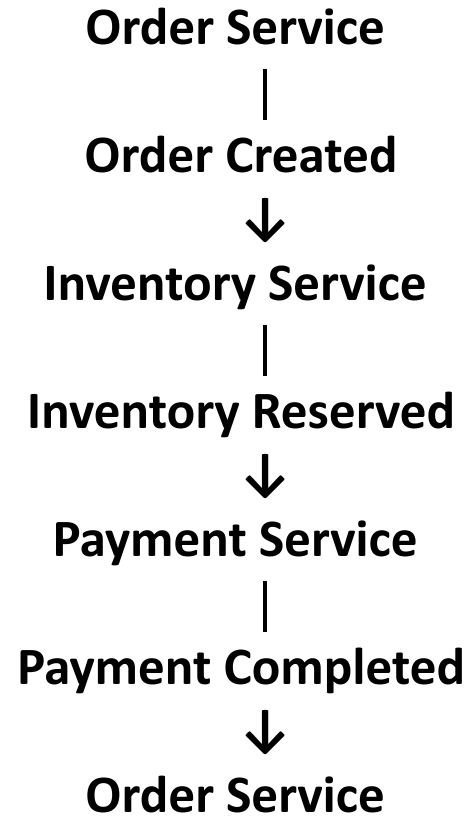
It is a new business operation that attempts to undo the effect of an earlier completed operation.

Saga with Failure : What
Heppens if any step get any
problem or issues ?



Types of Saga

- There are two major ways to coordinate a Saga:
- A. Choreography
 - In Choreography, there is no central coordinator.
- Each service:
 - Performs its local transaction.
 - Publishes an event.
 - Reacts to events from other services.



Advantages

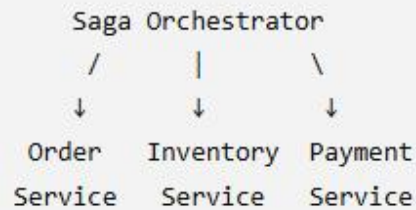
- No central coordinator
- Loosely coupled
- Suitable for simple workflows

Disadvantages

- Difficult to understand when many services are involved
- Business workflow becomes distributed
- Debugging can become difficult

Orchestration

In **Orchestration**, a central **Saga Orchestrator** controls the sequence of operations.



The orchestrator tells each service what action to perform and handles the next step or compensation.

Advantages

- Centralized workflow
- Easier to understand complex transactions
- Easier to monitor and manage

Disadvantages

- Orchestrator adds complexity
- More central coordination
- Orchestrator must be properly managed

Choreography

No central coordinator

Services react to events

Decentralized

Suitable for simpler workflows

Can become difficult to trace

More event-driven

Orchestration

Uses central orchestrator

Orchestrator directs services

Centralized workflow

Suitable for complex workflows

Easier to visualize workflow

More command-driven

Saga and Eventual Consistency

- Saga generally works with eventual consistency.
- Consider:
 - Order DB → Order Created
 - Inventory DB → Inventory Reserved
 - Payment DB → Payment Completed
- These databases do not necessarily update at exactly the same time.
- Therefore:
 - Saga sacrifices immediate global consistency in favor of independent local transactions and eventual consistency.

Advantages of Saga

- 1. Avoids Large Distributed Transactions
- 2. Supports Independent Services
- 3. Better Availability
- 4. Supports Eventual Consistency
- 5. Failure Recovery

Disadvantages

- More complex application logic
- Temporary inconsistent states
- Compensation logic must be carefully designed
- Debugging distributed workflows is difficult
- Failures can occur during compensation itself
- Long-running Sagas can be difficult to manage

2.6 Microservices with Asynchronous Messaging

A communication mechanism

Introduction

- Asynchronous Messaging is a communication mechanism in which a microservice sends a message to another service without waiting for an immediate response.
- The sender can continue its work after sending the message, while the receiver processes the message when it is ready.
- This is useful in MSA because microservices are independently deployable and loosely coupled.

Synchronous vs Asynchronous Communication

- Synchronous Communication

The sender waits for a response.

Service A — Request —→ Service B

Service A ← Response — Service B



Waits for response

- Asynchronous Communication

The sender sends a message and continues without waiting.

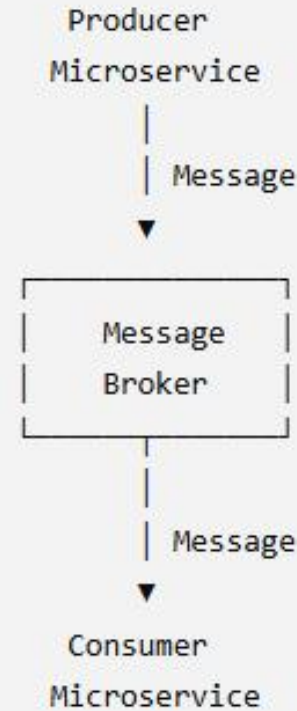
Service A — Message —→ Message Broker



Service B

Basic Architecture :

A **Message Broker** is commonly used between the producer and consumer.



Main Components

Producer: Creates and sends the message.

Message Broker: Receives, stores/routes, and delivers messages.

Consumer: Receives and processes the message.

Message: Contains the information/event being communicated.

Working of Asynchronous Messaging

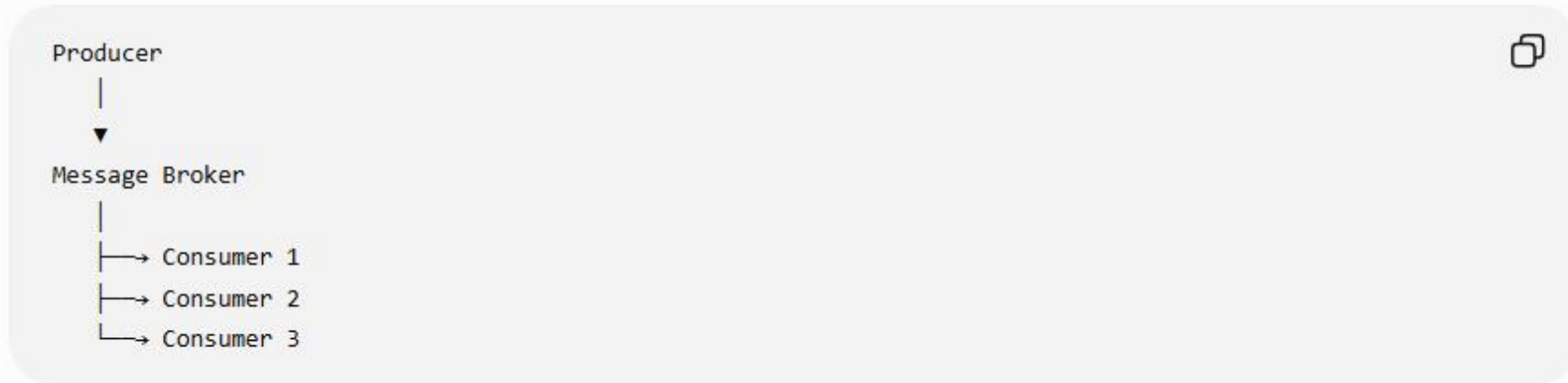
- Step 1: Order Creation
 - The Order Service creates an order.
- Step 2: Message Generation
 - It generates an event such as:
 - OrderCreated
- Step 3: Message Delivery
 - The event is sent to the message broker.
- Step 4: Consumers Process Message
 - Different services consume the message according to their requirements.
- Step 5: Independent Processing
 - Each service performs its own processing without requiring the Order Service to wait.

Why Use Asynchronous Messaging in MSA?

- 1. Loose Coupling
- 2. Better Availability
- 3. Improved Scalability
- 4. Better Performance
- 5. Event-Driven Architecture

Message Broker

A **Message Broker** acts as an intermediary between communicating services.



The broker helps with:

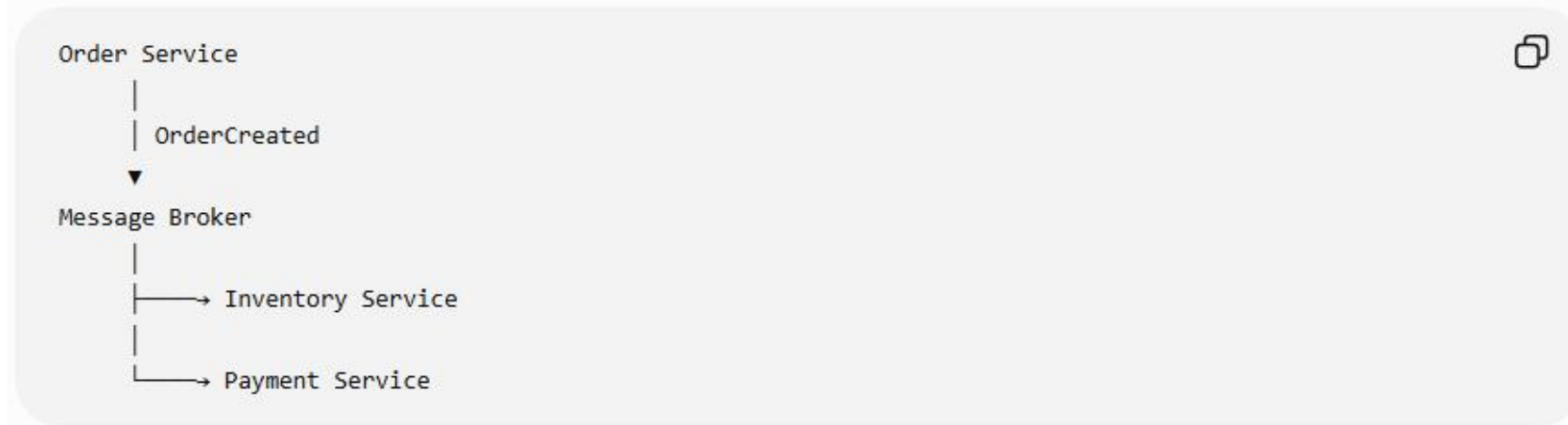
- Message routing
- Message buffering
- Delivery
- Decoupling producers and consumers
- Supporting asynchronous communication

Apache Kafka is an important example of a distributed event-streaming platform used for this type of communication and is covered in the next topics.

Asynchronous Messaging and Eventual Consistency

Asynchronous messaging is closely related to **eventual consistency**.

For example:



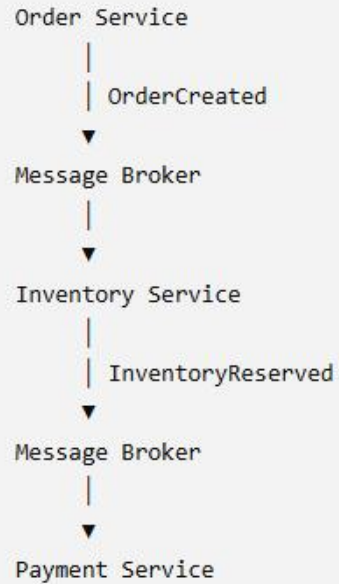
Inventory and Payment may process the event at different times.

Therefore, the data in different services may temporarily differ but can become consistent after the messages are processed.

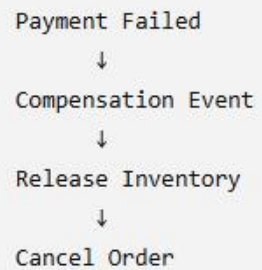
Asynchronous Messaging → Event Processing → Eventual Consistency

8. Asynchronous Messaging in Saga

Asynchronous messaging can also be used to coordinate a **Saga**.



If payment fails, another message can initiate a compensating action.



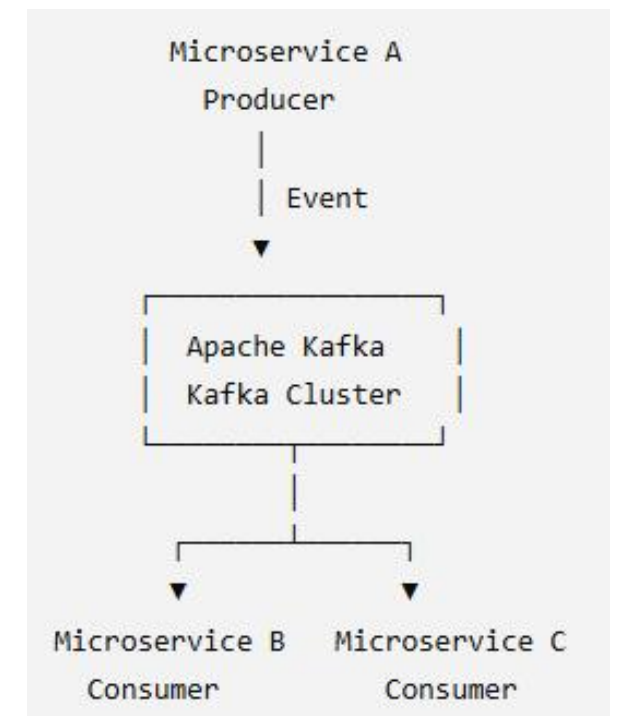
Thus, messaging provides a mechanism for services to coordinate distributed business operations.

2.7 Working with Apache Kafka

distributed messaging/event-streaming platform

What is Apache Kafka?

- Apache Kafka is a distributed messaging/event-streaming platform used for transferring and processing messages or events between applications and services.
- In MSA, Kafka can act as a communication layer between microservices.



Basic Components of Kafka

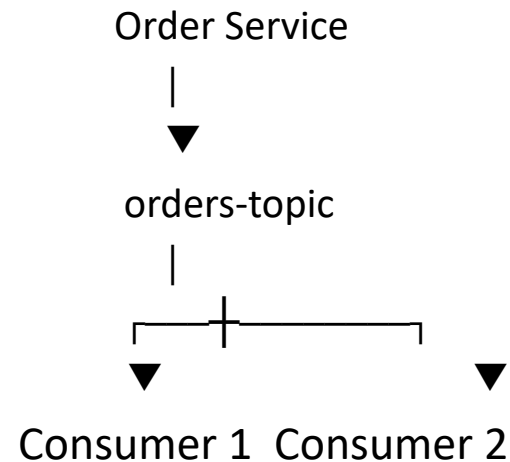
- A. Producer
 - A Producer is an application or service that publishes messages/events to Kafka.
 - Example :
 - Order Service → OrderCreated Event → Kafka
- B. Consumer
 - A Consumer reads and processes messages from Kafka.
 - Example:
 - Kafka → Payment Service

Basic Component of Kafka

- C. Broker
- A Broker is a Kafka server responsible for storing and serving messages.
- Important points from your PPT:
 - Brokers provide lightweight and fast message storage.
 - Messages are written to disk.
 - Kafka uses the operating system's page cache.
 - Data is stored in files organized by topic and partition.
 - Brokers handle replication.
 - Brokers use relatively little memory.
 - Brokers handle data retention/TTL operations.

3. Topic

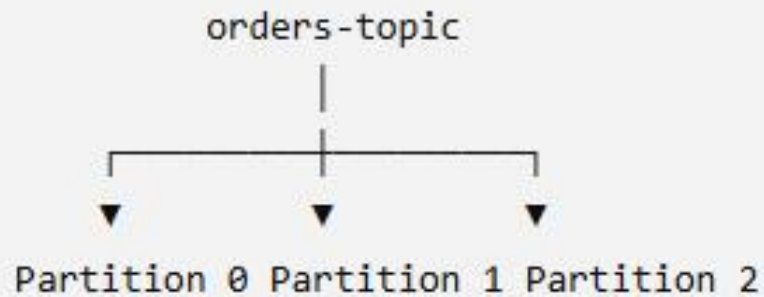
- A Topic is a named category in Kafka to which producers publish messages and from which consumers read messages.
- Example:



- For example:
 - orders
 - payments
 - inventory
 - notifications

4. Partition

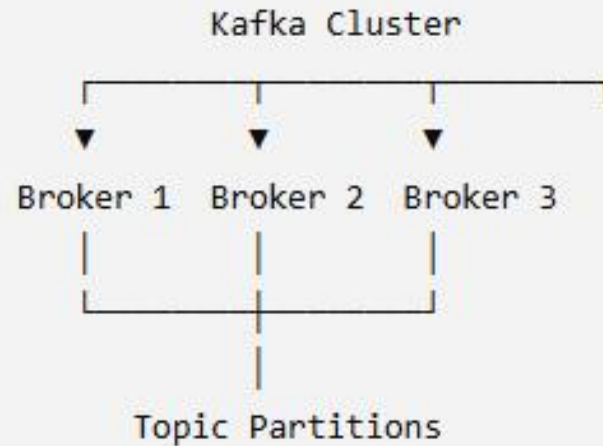
A Kafka topic can be divided into multiple **partitions**.



Partitions allow Kafka to distribute data and support parallel processing.

5. Kafka Cluster

A Kafka system can contain multiple brokers working together as a **cluster**.



Kafka uses replication so that data can remain available even if a broker fails.

6. Working of Kafka

- The basic flow is:

Producer

|

| 1. Publish Event



Topic

|

| 2. Stored in Partition



Kafka Broker

|

| 3. Consumer Reads



Consumer

|

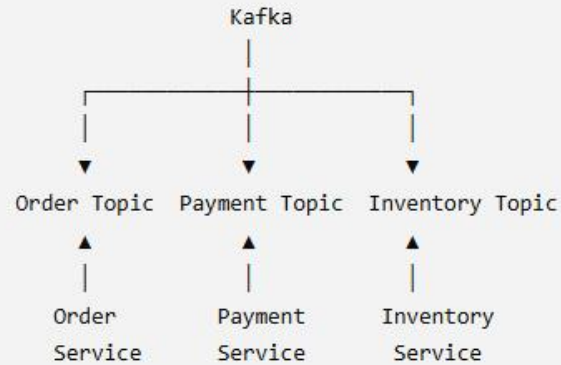


Business Process_{ing}

- Step 1 – Producer Creates Event
 - For example:
 - OrderCreated
- Step 2 – Producer Sends Event
 - The event is sent to a Kafka topic.
- Step 3 – Kafka Stores Event
 - The broker stores the event in the appropriate topic partition.
- Step 4 – Consumer Reads Event
 - The consumer reads the event from Kafka.
- Step 5 – Consumer Processes Event
 - The receiving microservice performs its required business operation.

7. Kafka in Microservices

Kafka can connect multiple independently deployed services.



For example:



This supports **event-driven microservices communication**.

8. Adding Producers

According to the PPT, adding producers involves:

1. Define the topic.
2. Define the required number of partitions using Kafka tools.
3. Optionally balance partition leaders across machines.
4. Start producing messages.

Define Topic



Define Partitions



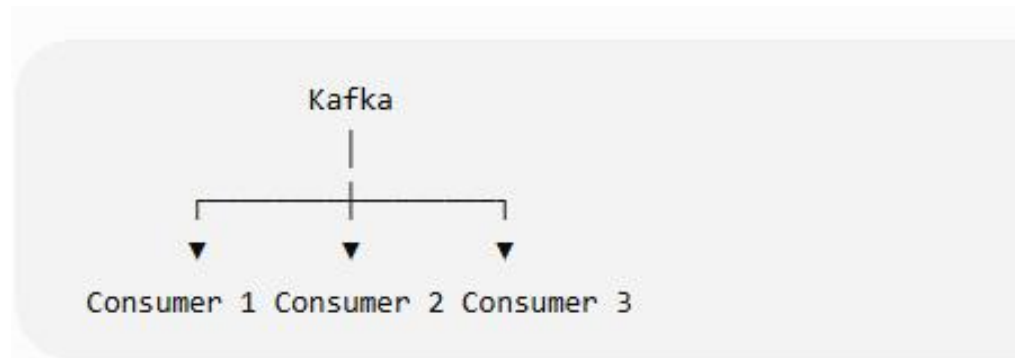
Configure Kafka



Start Producing

9. Adding Consumers

- Consumers can be added without significantly affecting producers.
- The PPT notes that adding consumers has little impact on producers and only a minor impact on brokers, mainly for maintaining connections.



This makes Kafka suitable for scalable event processing.

10. Kafka Retention

- Kafka stores messages on disk rather than requiring them to remain only in memory.
- The broker manages data retention and can remove data when it becomes too old according to the configured retention policy. Your PPT describes this as handling TTL operations and dropping old files.
- This makes it possible for consumers to process messages later rather than requiring them to be online at the exact time a message is produced

Advantages

- High Performance
- Scalability
- Fault Tolerance
- Loose Coupling
- Asynchronous Communication
- Persistent Storage
- Suitable for Event-Driven MSA

2.8 Kafka Commands

Practical Portion

1. Create a Kafka Topic

- A topic is created using the Kafka topic command.
- `kafka-topics.sh --create --topic orders \`
 - `--bootstrap-server localhost:9092 \`
 - `--partitions 3 \`
 - `--replication-factor 1`

 **Bash**



```
kafka-topics.sh --create --topic orders \  
--bootstrap-server localhost:9092 \  
--partitions 3 \  
--replication-factor 1
```

Important Options

Option	Meaning
<code>--create</code>	Creates a new topic
<code>--topic</code>	Specifies topic name
<code>--bootstrap-server</code>	Specifies Kafka broker
<code>--partitions</code>	Number of partitions
<code>--replication-factor</code>	Number of replicas

Example

```
Topic: orders  
Partitions: 3  
Replication Factor: 1
```



2. List Kafka Topics

- To display all available topics:
- `kafka-topics.sh --list \`
- `--bootstrap-server localhost:9092`
- Purpose: Displays the topics available in the Kafka broker.

3. Describe a Topic

- To view information about a particular topic:

- `kafka-topics.sh --describe \`
- `--topic orders \`
- `--bootstrap-server localhost:9092`

- It can provide information such as:

- Topic name
- Number of partitions
- Replication factor
- Partition information
- Leader and replica information

4. Start a Kafka Producer

- A producer is used to send messages to a topic.
- `kafka-console-producer.sh \`
- `--topic orders \`
- `--bootstrap-server localhost:9092`
- After starting the producer, messages can be entered from the console.
- `> Order 101`
- `> Order 102`
- `> Order 103`
- These messages are published to the orders topic.

5. Start a Kafka Consumer

- A consumer reads messages from a topic.
- `kafka-console-consumer.sh \`
- `--topic orders \`
- `--bootstrap-server localhost:9092`
- To consume messages from the beginning:
 - `kafka-console-consumer.sh \`
 - `--topic orders \`
 - `--bootstrap-server localhost:9092 \`
 - `--from-beginning`

6. Delete a Topic

- A topic can be deleted using:
- `kafka-topics.sh --delete \`
- `--topic orders \`
- `--bootstrap-server localhost:9092`
- Topic deletion must be enabled/configured appropriately in the Kafka environment.

7. Important Commands at a Glance

- **Operation**

- Create Topic
- List Topics
- Describe Topic
- Delete Topic
- Start Producer
- Start Consumer
- Read from Beginning

- **Command**

kafka-topics.sh --create

kafka-topics.sh --list

kafka-topics.sh --describe

kafka-topics.sh --delete

kafka-console-producer.sh

kafka-console-consumer.sh

--from-beginning

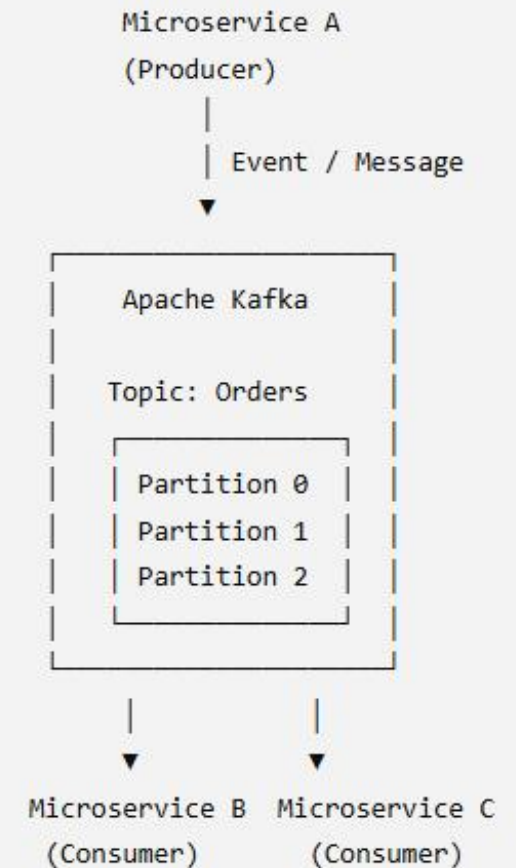
2.9 Creating MSA with Kafka

Important for Practical + Theory Aspect

1. Introduction

- Creating Microservices Architecture (MSA) with Kafka means using Apache Kafka as the communication layer between independent microservices.
- Instead of one service directly calling another service synchronously, a service can:
 - Produce a message/event.
 - Send it to a Kafka topic.
 - Kafka stores the message.
 - One or more consumer services read the message.
 - Consumers perform their own business operations.

Basic Architecture



2. Main Components

- **Component**

- Producer
- Kafka Broker
- Topic
- Partition
- Consumer
- Consumer Group
- Kafka Cluster

- **Role**

Publishes messages/events to Kafka

Stores and manages messages

Logical category in which messages are stored

Divides a topic for scalability

Reads and processes messages

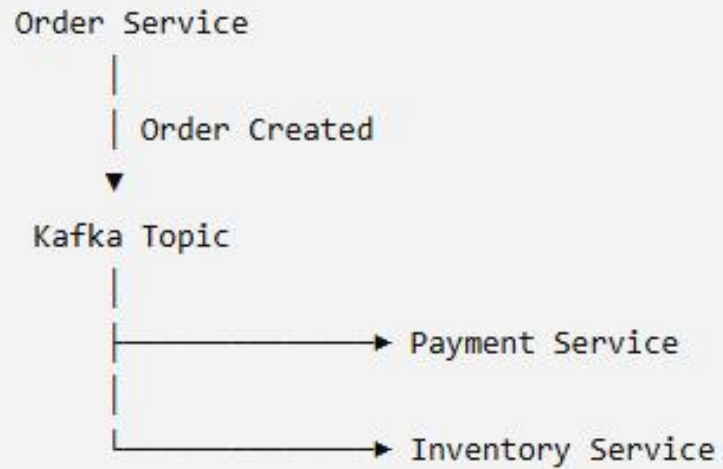
Group Allows multiple consumers to share processing

Collection of Kafka brokers

3. How MSA Uses Kafka

Consider an **Online Shopping System**.

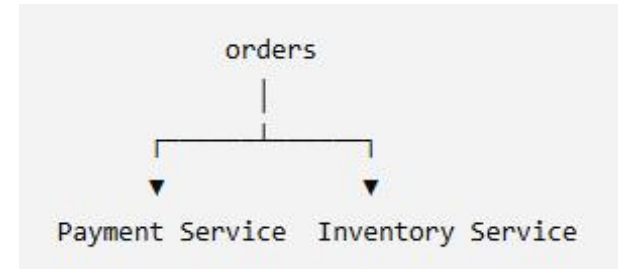
There may be three independent microservices:



- Step-by-step
- Step 1 – Order Service receives request
 - The customer places an order.
 - Customer → Order Service
 - Order Service creates the order in its own database.
- Step 2 – Order Service publishes event
 - After creating the order, it publishes:
 - OrderCreated
 - to the Kafka topic:
 - orders
- Step 3 – Kafka stores the event
- Kafka receives and stores the message inside the appropriate topic partition.
- Order Service
 - |
 - ▼
 - Kafka Producer
 - |
 - ▼
 - orders Topic

Step 4 – Consumer services receive event

- Payment Service and Inventory Service can consume the event.



- The services process the event independently.
- Step 5 – Services perform local transactions
- Each service updates its own database.
- Payment Service → Payment DB
- Inventory Service → Inventory DB
- This fits the MSA principle where each service owns its data rather than depending on a central database.

4. Producer Microservice

- A Producer Microservice is responsible for publishing events/messages to Kafka.

- Example:

- Order Service

- |

- | OrderCreated

- ▼

- Kafka Producer

- |

- ▼

- orders Topic

The producer does not need to know the internal implementation of the consumer.

It only needs to know:

Kafka broker address

Topic name

Message/event format

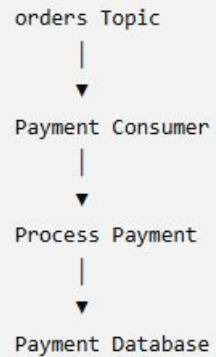
Example Event

```
{  
  "orderId": 101,  
  "customerId": 25,  
  "amount": 1500,  
  "status": "CREATED"  
}
```

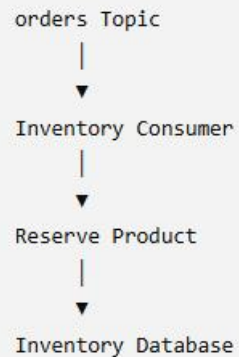
5. Consumer Microservice

A Consumer Microservice reads messages from Kafka and performs its business logic.

Example:



Another consumer can independently process the same business event:



6. Creating MSA with Kafka – Steps

- Step 1: Identify Microservices
- Divide the application into independent services.
- Example:
 - Order Service
 - Payment Service
 - Inventory Service
 - Notification Service

- Step 2: Start Kafka
- Run the Kafka broker/cluster.
- Kafka Cluster
 - |
 - └─ Broker
- Step 3: Create Topics
- Create topics according to business events.
- Example:
 - orders
 - payments
 - inventory
 - notifications
- The PPT states that topics and their number of partitions are defined using Kafka tools before producers start producing.

- Step 4: Configure Producer
- Configure the Order Service to publish messages to:
- orders

- Step 5: Configure Consumers
- Configure:
- Payment Service → orders
- Inventory Service → orders

- Step 6: Publish Event
- Order Service publishes:
- OrderCreated

Step 7: Process Event

Consumers receive and process the event.

Order Service



OrderCreated



Kafka

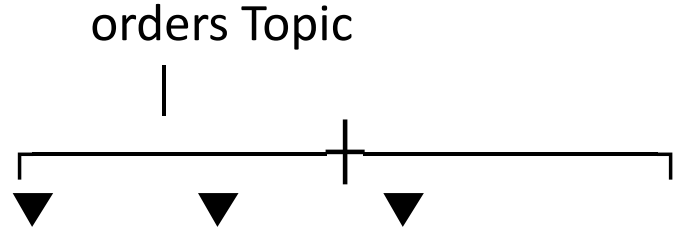


Payment Service

Inventory Service

7. Kafka Topic and Partitions

A topic can be divided into multiple partitions.



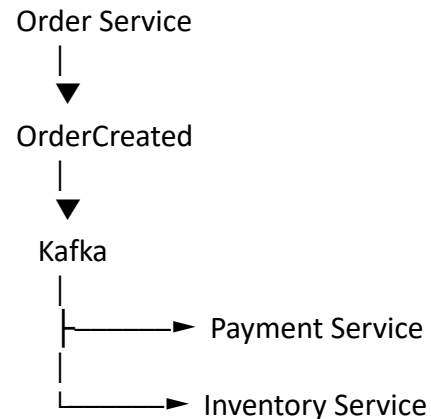
Partition 0 Partition 1 Partition 2

Partitions provide:

- Parallel processing
- Scalability
- Distribution of messages across brokers
- Higher throughput

Kafka + MSA + Eventual Consistency

- Kafka is commonly used to implement asynchronous communication between services.
- Consider:



- The Order Service may complete first, while Payment and Inventory update their own databases shortly afterward.
- Therefore, the system may temporarily have different states.
- This is related to eventual consistency, which the PPT describes as allowing temporary inconsistency until distributed data becomes consistent.

Kafka + Saga

- Example:

Order Created



Kafka



Payment Service

Payment Success



Kafka



Inventory Service



Inventory Reserved



Kafka



Order Confirmed

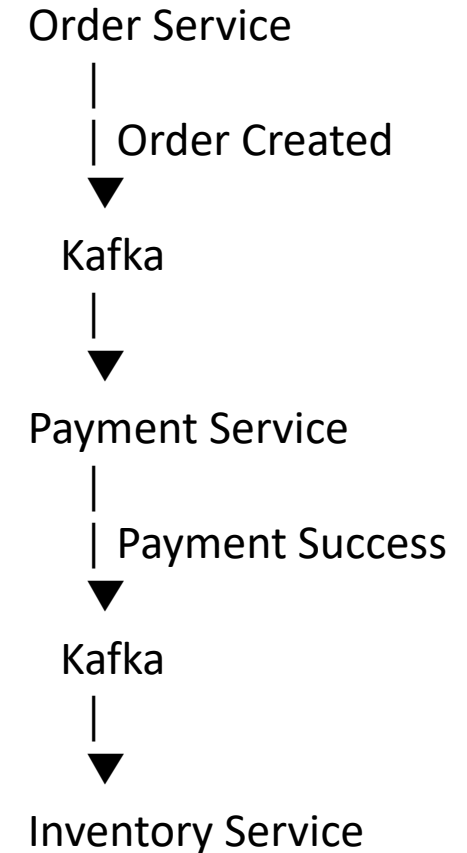
if a later step fails, a compensating action can be triggered.

2.10 Handling Transaction with Kafka

multiple services and separate databases

Introduction

- In Microservices, a single business transaction may involve multiple services and separate databases. Therefore, maintaining consistency using one traditional ACID transaction is difficult.
- Kafka can be used to coordinate transactions through messages/events between services.
- Each service performs its own local transaction and then publishes an event to trigger the next service. This is the basic idea behind using Saga with messaging



Transaction Flow

- Example: Place Order
- Order Service creates the order.
- It publishes OrderCreated to Kafka.
- Payment Service consumes the event and processes payment.
- Payment Service publishes PaymentCompleted.
- Inventory Service consumes the event and reserves the product.
- If a step fails, a compensating transaction can be triggered.

Why Not Use Distributed Transactions?

- A traditional distributed transaction such as 2-Phase Commit (2PC) provides strong consistency, but it has disadvantages:
- Complex to maintain
- High latency
- Lower throughput
- Blocking
- Possible deadlocks
- Transaction coordinator can become a single point of failure
- Therefore, MSA commonly uses local transactions + Kafka events + Saga/eventual consistency instead.

2.11 CQRS

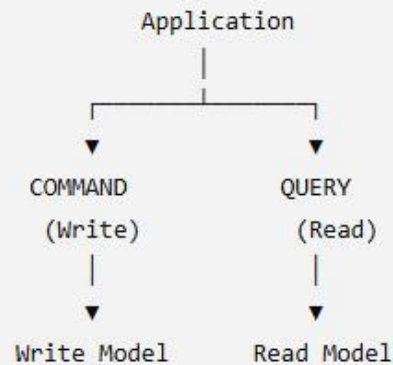
Command Query Responsibility Segregation

CQRS

- CQRS (Command Query Responsibility Segregation) is a design pattern that separates data modification operations (Commands) from data reading operations (Queries).

1. Basic Concept

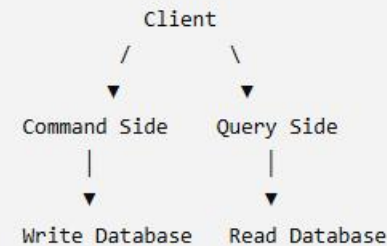
Instead of using the same model for both reading and writing:



- **Command** → Creates, updates, or deletes data.
- **Query** → Only retrieves data; it does not modify data.

2. CQRS in MSA

In microservices, the write and read sides can be scaled independently.



For example, in an **Order System**:

- `CreateOrder` → Command
- `UpdateOrder` → Command
- `GetOrderDetails` → Query
- `GetOrderHistory` → Query

- 3. Advantages
 - Separates read and write responsibilities.
 - Read and write sides can be scaled independently.
 - Read models can be optimized for fast queries.
 - Useful for complex microservice systems.
 - Can work well with event-driven architecture.
- 4. Limitation
 - CQRS increases system complexity because separate read and write models/databases may need to be maintained.

2.12 Introduction to Microstreams

Small But Important

Introduction

- Microstreams refers to handling small, continuous streams of data/events between microservices. It fits naturally with event-driven and asynchronous architectures.
- Instead of processing a large batch of data at once, data is processed as a continuous sequence of small.
- Processes data/events continuously.
- Supports real-time or near-real-time processing.
- Useful for event-driven microservices.
- Helps services communicate asynchronously.
- Kafka can be used to transport and process such event streams.

Basic Idea

Instead of processing a large batch of data at once, data is processed as a **continuous sequence of small events**.



Example

In an e-commerce system:



Each event can be processed as it arrives rather than waiting for the complete dataset.

2.13 Design Patterns for MSA

Design patterns for Microservices Architecture (MSA)

- Design patterns for Microservices Architecture (MSA) are reusable solutions for common problems encountered while designing, communicating, deploying, testing, and monitoring microservices.

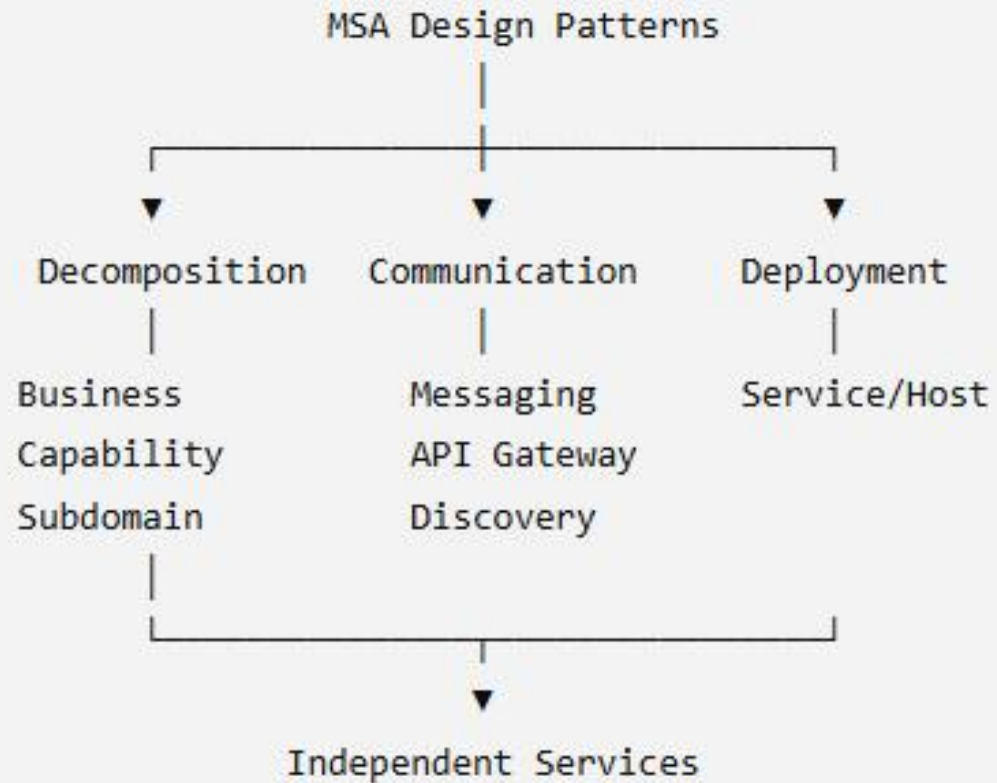
- Important MSA Design Patterns

- **Category**

- **Important Patterns**

- | | |
|---------------------|---|
| • Decomposition | Business Capability, Subdomain |
| • Data Management | Database per Service |
| • Communication | Messaging, Remote Procedure Invocation |
| • Access | API Gateway |
| • Service Discovery | Client-side Discovery, Server-side Discovery |
| • Deployment | Single Service per Host, Multiple Services per Host |
| • Security | Access Token |
| • Fault Handling | Circuit Breaker |
| • Observability | Log Aggregation, Metrics, Distributed Tracing, Health Check |
| • Testing | Service Component Test, Integration Contract Test |
| • UI | Server-side / Client-side UI Composition |

Simple / Basic Architecture



- 1. Database per Service
 - Each microservice owns its own database, helping maintain loose coupling.
- 2. API Gateway
 - Provides a common entry point through which clients access microservices.
- 3. Messaging
 - Services communicate asynchronously through messages/events.
- 4. Service Discovery
 - Helps locate available instances of a service.
- 5. Circuit Breaker
 - Helps prevent failures in one service from continuously affecting other services.
- 6. Observability Patterns
 - Logging, metrics, tracing and health checks help monitor distributed services.
 - The reference book specifically identifies Database per Service, API Gateway, Discovery, Messaging, deployment, Circuit Breaker, and observability patterns as related patterns to microservices

Thank you

Vraj Suratwala

Department of ICT, VNSGU